

Sequence Diagrams

1. What Is a Sequence Diagram?

A **sequence diagram** is a UML interaction diagram that shows how a group of objects work together to carry out one scenario, by showing the **messages** they send to each other **in time order**. The participants are drawn across the top, each with a dashed vertical line called a **lifeline**, and time runs from top to bottom, so reading the diagram downwards tells you exactly who asks whom to do what, and in which order. For example, a sequence diagram for “Withdraw Cash” shows the customer inserting a card, the ATM asking the bank to verify the PIN, the bank replying, and the cash dispenser releasing the money. A sequence diagram usually describes one path through one use case, so a system normally has several of them. It helps designers decide which object is responsible for each step, shows developers which methods each class needs and what they return, and gives testers a clear script of the expected interaction.

2. Basic Notation

Element	How it is drawn	Meaning
Actor	Stick figure at the top	A person or external system that starts or takes part in the interaction.
Lifeline	Box with the participant’s name, and a dashed line going down	One participant (usually an object) and its existence over time.
Activation bar	Thin rectangle on a lifeline	The period during which the object is busy carrying out an operation. Also called an execution specification.
Message	Arrow from one lifeline to another	A request, signal or reply sent between participants. Arrow style shows the kind (see Section 3).
Combined fragment	Rectangle with an operator such as alt in the top-left corner	A group of messages that is optional, repeated, or one of several alternatives (see Section 6).

Naming lifelines. A lifeline is labelled `objectName : ClassName`. The object name is optional, so `:ATM` means “some object of class ATM” and `atm1 : ATM` means a specific object called `atm1`. Actors are labelled with their role, such as Customer.

Naming messages. A message is labelled like a method call: `name(arguments)`, for example `verifyPIN(cardNo, pin)`. A message to an object normally becomes a method of that object’s class, so `verifyPIN` becomes an operation of the Bank class. Reply messages are labelled with the value returned, such as `valid` or `result`.

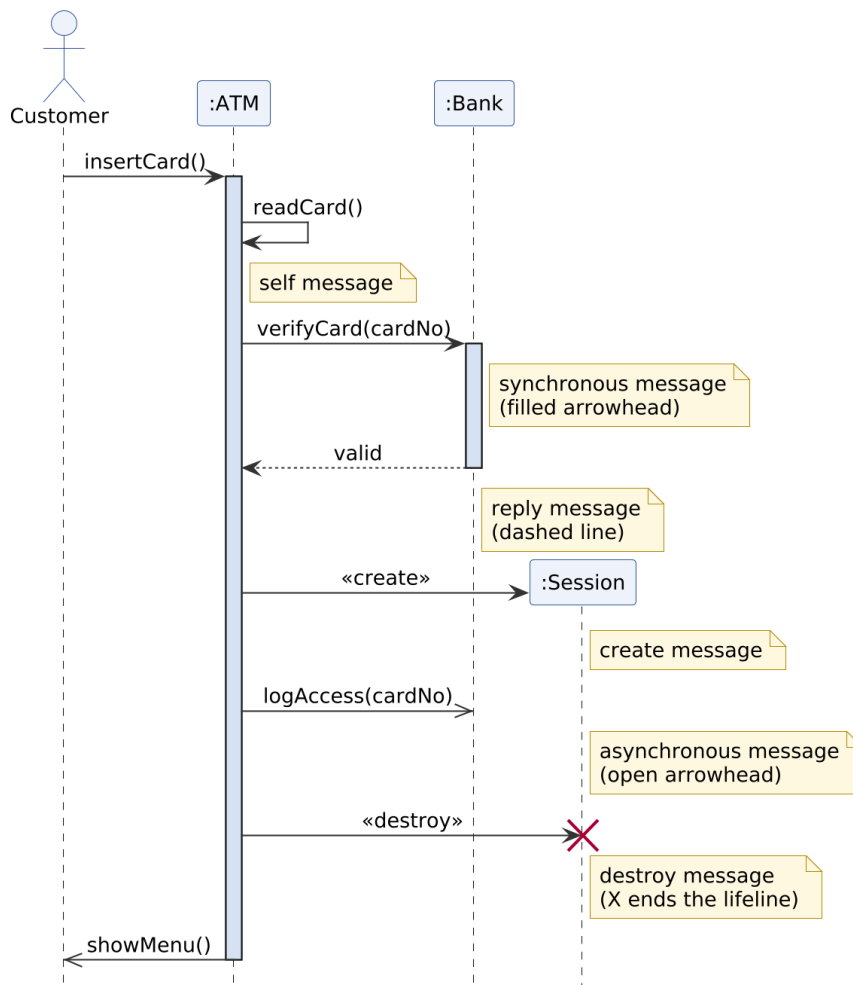


Figure 1. The main notation elements of a sequence diagram.

3. Types of Messages

Message	How it is drawn	Meaning
Synchronous	Solid line, filled arrowhead	The sender waits until the receiver finishes and replies. This is an ordinary method call.
Asynchronous	Solid line, open arrowhead	The sender continues without waiting, e.g. sending a notification or displaying something to the user.
Reply (return)	Dashed line, open arrowhead	The answer returned to the caller of a synchronous message.
Self message	Arrow that leaves and returns to the same lifeline	An object calling one of its own operations.
Create	Dashed or solid arrow pointing at the new object's box, labelled «create»	Creates a new object. Its lifeline starts lower down, at the point of creation.
Destroy	Arrow ending in a large X, labelled «destroy»	Ends an object's life. Its lifeline stops at the X.
Found / Lost	Arrow starting from, or ending at, a small filled circle	A message from an unknown sender (found), or one that never reaches a receiver (lost).

4. How Is It Different from Other Diagrams?

A **use case diagram** shows *what* users can do with the system, such as “Withdraw Cash”, but not how it happens; a sequence diagram takes one of those use cases and shows *how* the objects inside the system cooperate step by step to achieve it. A **class diagram** is static: it shows which classes exist, their attributes and operations, and how they are related, while a sequence diagram shows those classes’ objects *in action* at run time, and every message in it should correspond to an operation in the class diagram. A **data flow diagram** follows data moving between processes and stores, with no notion of objects, method calls or strict order, whereas a sequence diagram is all about objects and the exact order of their messages. The comparison that matters most is with the **state chart diagram**: a state chart follows *one object* across *many scenarios* over its whole life, showing all the states it can be in, while a sequence diagram follows *many objects* through *one scenario*, showing the messages between them. Put simply, a state chart asks “how does this one object behave in every situation?” and a sequence diagram asks “who talks to whom, and in what order, to get this particular job done?”

5. Reading Order and Activation

Always read a sequence diagram from **top to bottom**: a message drawn lower happens later. Horizontal position has no meaning in terms of time; it only separates participants. By convention the actor who starts the scenario is placed on the far left, and participants are arranged left to right in roughly the order they get involved, which keeps arrows short and easy to follow.

An **activation bar** starts when an object receives a message and ends when it has finished handling it. For a synchronous call, the caller’s bar continues while it waits, and the receiver’s bar ends at the reply. Activation bars make nesting visible: if the ATM calls the Bank while handling `insertCard()`, the Bank’s bar sits inside the time span of the ATM’s bar.

6. Combined Fragments

Real scenarios contain decisions and repetition. A **combined fragment** is a rectangle drawn over part of the diagram, with an **interaction operator** in its top-left corner that says how the enclosed messages behave. The fragment is divided into one or more **operands** by dashed horizontal lines, and each operand can have a **guard** in square brackets. An operand runs only if its guard is true, exactly like guards in a state chart.

Operator	Meaning	Example
alt	Alternatives: like if / else. Exactly one operand runs, the one whose guard is true. Use [else] for the last operand.	[pin valid] / [pin invalid]
opt	Option: like an if without else. The single operand runs only if its guard is true.	[receiptRequested]
loop	Repetition: the operand repeats while its guard is true, or a given number of times.	loop(1,3) [pin invalid]
par	Parallel: the operands run at the same time.	Print receipt while dispensing cash

Operator	Meaning	Example
break	Break: if the guard is true, the operand runs and the rest of the enclosing interaction is skipped.	[card blocked]
ref	Reference (interaction use): the box stands for another sequence diagram, drawn separately.	ref Authenticate Customer

Fragments can be nested. In Figure 2 an opt fragment sits inside the first operand of an alt fragment, so a receipt is only printed on the successful path.

7. Worked Example: ATM “Withdraw Cash”

Figure 2 shows the main scenario of the Withdraw Cash use case. It uses the same ATM as the state chart handout, but now looks at how the ATM cooperates with the bank and the cash dispenser.

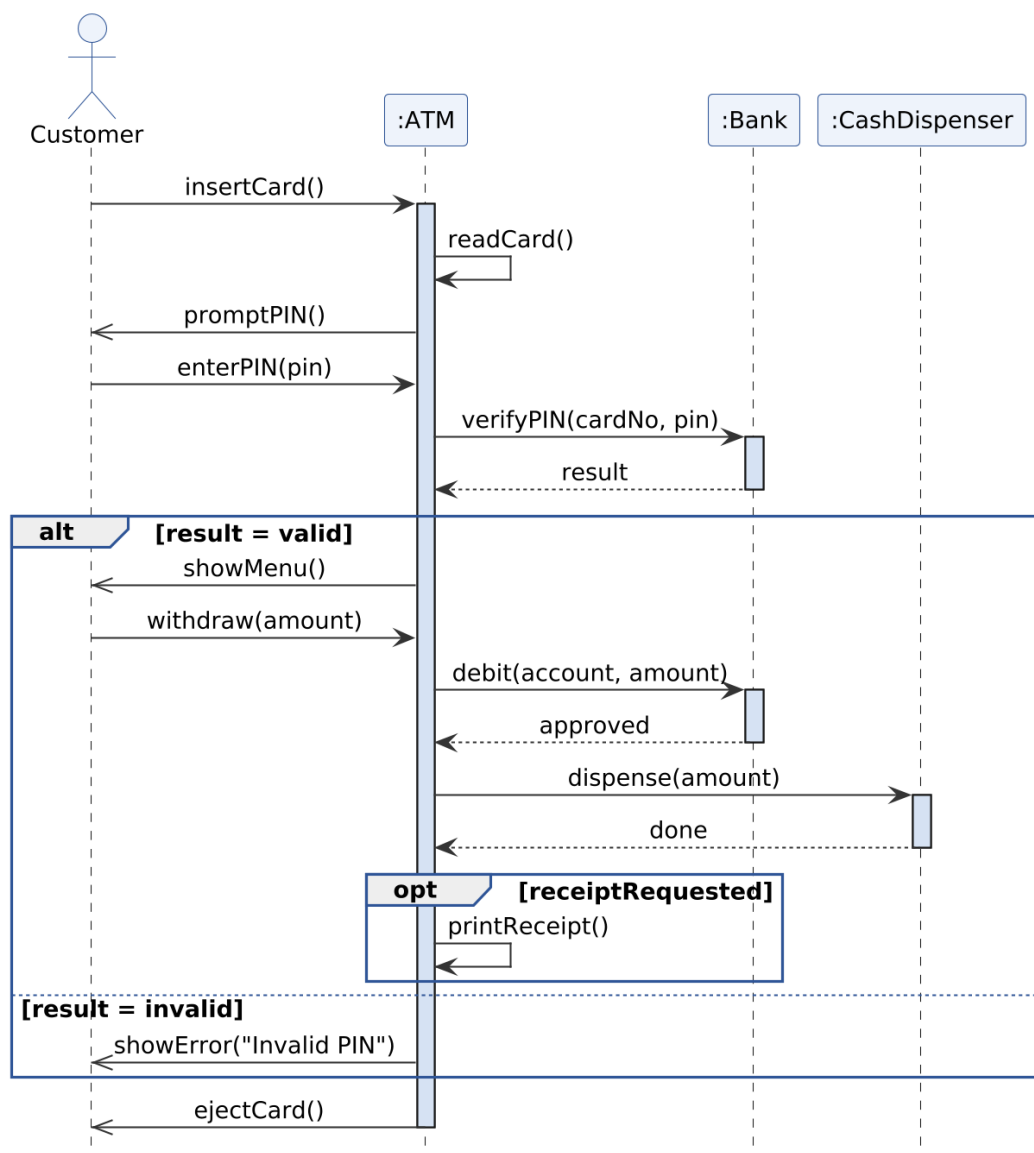


Figure 2. Sequence diagram for the ATM Withdraw Cash scenario.

How to read it

- The **Customer** inserts a card. The ATM becomes active and reads the card with a self message, `readCard()`.
- The ATM asks for the PIN with an asynchronous message, `promptPIN()`, and the customer replies with `enterPIN(pin)`.
- The ATM makes a synchronous call, `verifyPIN(cardNo, pin)`, and waits. The Bank returns result on a dashed reply arrow.
- The alt fragment chooses a path. If `[result = valid]`, the customer selects an amount, the Bank debits the account, and the CashDispenser releases the cash. Inside this path, the opt fragment prints a receipt only if `[receiptRequested]`.
- If `[result = invalid]`, the ATM shows an error instead.
- Both paths join again below the fragment, and the ATM ejects the card.
- Every message received by an object suggests an operation for its class: Bank needs `verifyPIN()` and `debit()`, CashDispenser needs `dispense()`.

8. Tips and Common Mistakes

- **One scenario per diagram.** Draw the main success path first; show variations with alt or opt, or draw a separate diagram for a very different path.
- **Start with the actor on the left** and arrange the other participants in the order they join in.
- **Match the class diagram.** Every message sent to an object should be an operation of that object's class, with matching parameters.
- **Use the right arrow.** Solid filled arrowheads for calls, open arrowheads for asynchronous messages, dashed lines only for replies. Do not label a reply as if it were a call.
- **Don't draw a reply for every call.** Show replies when the returned value matters to the story; otherwise they add clutter.
- **Keep it readable.** If a diagram grows beyond a page, move a sub-scenario into its own diagram and refer to it with a ref fragment.
- **Don't use it for algorithms.** Detailed logic inside one method belongs in an activity diagram or in code, not in a sequence diagram.

9. Step-by-Step: Drawing Figure 2 in StarUML

Follow these steps in order to reproduce Figure 2. Menu names may differ slightly between StarUML versions; if an option is not where described, right-click the element and look in the **Add** submenu, or double-click it to open **Quick Edit**.

Part A — Create the model elements

- 1 Open StarUML and create a new project with **File → New**.
- 2 In the **Model Explorer** (top right), right-click **Model** and choose **Add → Actor**. Name it Customer.
- 3 Right-click **Model** again and choose **Add → Class**. Name it ATM. Repeat to create the classes Bank and CashDispenser.

- 4 Right-click **Model** and choose **Add Diagram → Sequence Diagram**. StarUML creates the diagram inside a new interaction. Rename the diagram to Withdraw Cash in the **Properties** panel (bottom right; turn it on from the **View** menu if it is hidden).
- 5 With the diagram selected, uncheck showSequenceNumber in the Properties panel if you do not want numbers such as “1:” in front of each message. Figure 2 does not show them.

Part B — Add the lifelines

- 1 Drag **Customer** from the Model Explorer and drop it at the top-left of the diagram. StarUML creates a lifeline with a stick figure.
- 2 Drag the **ATM** class from the Model Explorer and drop it to the right of Customer. StarUML creates a lifeline of type ATM.
- 3 Repeat for **Bank** and **CashDispenser**, placing them left to right in that order.
- 4 Check the labels. Each should read :ATM, :Bank and :CashDispenser. If StarUML added a generated name in front (for example Lifeline1: ATM), double-click the lifeline and delete the name, leaving only :ATM.
- 5 Drag each lifeline’s bottom end downwards so there is plenty of room for all the messages.

Part C — How to draw each kind of message

- **Ordinary message:** select **Message** in the Toolbox, press on the sender’s lifeline and drag to the receiver’s lifeline. Double-click the arrow and type the label, for example verifyPIN(cardNo, pin).
- **Message kind:** select the message and set messageSort in the Properties panel: synchCall for a synchronous call, asynchSignal for an asynchronous message, reply for a reply.
- **Reply:** double-click the synchronous call and choose **Add Reply Message**. StarUML draws a dashed arrow back to the caller; double-click it and type the returned value, for example result.
- **Self message:** select **Self Message** in the Toolbox and click on the lifeline.
- **Order:** draw messages from top to bottom in the order they happen. StarUML adds activation bars automatically; drag the end of a bar to lengthen or shorten it if needed.

Part D — Draw the messages in this order

#	From	To	Label	messageSort
1	Customer	:ATM	insertCard()	synchCall
2	:ATM	:ATM	readCard()	Self Message tool
3	:ATM	Customer	promptPIN()	asynchSignal
4	Customer	:ATM	enterPIN(pin)	synchCall
5	:ATM	:Bank	verifyPIN(cardNo, pin)	synchCall
6	:Bank	:ATM	result	reply (Add Reply Message on #5)
7	:ATM	Customer	showMenu()	asynchSignal
8	Customer	:ATM	withdraw(amount)	synchCall
9	:ATM	:Bank	debit(account, amount)	synchCall
10	:Bank	:ATM	approved	reply (Add Reply Message on #9)
11	:ATM	:CashDispenser	dispense(amount)	synchCall
12	:CashDispenser	:ATM	done	reply (Add Reply Message on #11)
13	:ATM	:ATM	printReceipt()	Self Message tool
14	:ATM	Customer	showError("Invalid PIN")	asynchSignal
15	:ATM	Customer	ejectCard()	asynchSignal

Leave some extra vertical space before message 7, between messages 13 and 14, and before message 15. The combined fragments will be drawn in these gaps.

Part E — Add the alt fragment

- 1 Select **Combined Fragment** in the Toolbox and drag a rectangle that starts just above message 7 and ends just below message 14, spanning from the Customer lifeline to the CashDispenser lifeline.
- 2 With the fragment selected, set `interactionOperator` to `alt` in the Properties panel. The label `alt` appears in the top-left corner.
- 3 Double-click the fragment and choose **Add Operand**, so the fragment has two operands separated by a dashed line.
- 4 Drag the dashed dividing line so it sits between message 13 and message 14. Messages 7–13 are now in the first operand and message 14 is in the second.
- 5 Double-click the first operand and set its **Guard** to `result = valid`. Double-click the second operand and set its Guard to `result = invalid`. StarUML shows each guard in square brackets; if you see double brackets, remove the ones you typed.

Part F — Add the opt fragment

- 1 Select **Combined Fragment** again and drag a smaller rectangle around message 13 (`printReceipt()`) only, inside the first operand of the alt fragment.

- 2 Set its interactionOperator to opt.
- 3 Double-click its operand and set the Guard to receiptRequested.
- 4 Check that message 15 (ejectCard()) lies below the alt fragment, outside both fragments, since it happens on both paths.

Part G — Check and export

- 1 Compare every label and arrow style with Figure 2 and the table in Part D. Replies should be dashed; asynchronous messages should have open arrowheads.
- 2 Optional: link each message to a real operation. Double-click a message and choose **Create Operation** (or **Select Operation** if it already exists). The operation is added to the receiver's class, keeping the sequence diagram and class diagram consistent.
- 3 Save the project with **File → Save**.
- 4 Export the picture with **File → Export Diagram As → PNG** (or SVG). This gives a clean image without the grid or the navigator thumbnail.